# Step 4 — A Timeout and a Partial-Result Fallback — Solution

> Back to the step · Hint 1 · Hint 2

**Story:** a run that's taking too long, or actually breaks, now returns a clearly-marked partial answer — the best draft reached so far — instead of hanging the caller indefinitely. **If not:** a tool that can hang forever with no timeout is one nobody will ever trust enough to build on.

## `mcp_server.py` (final) — the async tool with a timeout

**Story:** the tool function itself is now declared `async`. The pipeline underneath is still **synchronous** code, so it runs inside `asyncio.to_thread(...)` — a plain blocking call wrapped directly in `wait_for` would block the whole event loop and never actually time out, defeating the point. `asyncio.wait_for` is what enforces the real timeout; on a `TimeoutError`, the `Progress` holder that background thread has been writing to the whole time gives you the best draft reached so far. **If not:** `await asyncio.wait_for(run_pipeline(...))` called directly on a sync function would hang exactly like before, timeout or not.

```
# mcp_server.py  (final)
import asyncio
import sys

from mcp.server.mcpserver import MCPServer

from src.config import load_config
from src.formatting import format_partial, format_result
from src.models.llm import get_model
from src.pipeline import Progress, run_pipeline
from src.utils.logger import get_logger

CONFIG = load_config()
MODEL = get_model(CONFIG)
logger = get_logger(__name__)

mcp = MCPServer("deep-research-service")


@mcp.tool()
```

```
async def deep_research(topic):
    """Research a topic through a fact-checked, multi-round pipeline; returns
    a verified summary plus a revision trace. Falls back to the best draft so
    far if it runs past the timeout."""
    logger.info(f"deep_research: {topic}")
    progress = Progress()
    try:
        result = await asyncio.wait_for(
            asyncio.to_thread(
                run_pipeline, MODEL, topic, CONFIG.max_revision_rounds,
                progress,
            ),
            timeout=CONFIG.timeout_seconds,
        )
        return format_result(result)
    except asyncio.TimeoutError:
        logger.warning(f"deep_research timed out on: {topic}")
        return format_partial(topic, progress)


if __name__ == "__main__":
    print("deep-research service starting", file=sys.stderr)
    mcp.run(transport="stdio")
```

Two things to understand here:

- **Why `to_thread`:** `run_pipeline` is ordinary synchronous code (blocking LLM calls in a loop). If you wrote `await asyncio.wait_for(run_pipeline(...))`, `run_pipeline(...)` would run to completion *before* `wait_for` ever got control — blocking the event loop, never timing out. `asyncio.to_thread` runs it on a worker thread so the event loop stays free and `wait_for` can actually fire.
- **Why the `Progress` holder works for the partial:** the thread writes `progress.latest_draft` and `progress.rounds` as it goes, and the holder is created *here*, so on timeout you read whatever the pipeline managed before the clock ran out. **Known limitation:** `to_thread` can't cancel the worker thread, so the pipeline keeps running in the background after the timeout — fine for a demo, but a real deployment would add a cancellation flag the loop checks each round.

## The integration test

```
# tests/integration/test_live_tool.py
# Launches the real deep-research MCP server and calls its tool over MCP.
# Needs a valid OPENAI_API_KEY. Run from the repo root:
#   python -m tests.integration.test_live_tool
import asyncio
```

```
import os

from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client


async def call_tool(topic, env=None):
    params = StdioServerParameters(
        command="python", args=["mcp_server.py"], env=env
    )
    async with stdio_client(params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()
            result = await session.call_tool("deep_research", {"topic": topic})
            return result.content[0].text


async def test_returns_trace_and_summary():
    text = await call_tool("Retrieval-Augmented Generation (RAG)")
    ok = "Revision trace" in text and "Summary:" in text
    result1 = "WRONG"
    if ok:
        result1 = "OK"
    print(f"case 1 (trace + summary) -> {result1}")


async def test_timeout_gives_partial():
    # a 0-second timeout forces the partial path even on a fast topic
    env = dict(os.environ)
    env["DEEP_RESEARCH_TIMEOUT_SECONDS"] = "0.001"
    text = await call_tool("anything", env=env)
    ok = "PARTIAL" in text
    result2 = "WRONG"
    if ok:
        result2 = "OK"
    print(f"case 2 (timeout -> PARTIAL) -> {result2}")


async def main():
    await test_returns_trace_and_summary()
    await test_timeout_gives_partial()


if __name__ == "__main__":
    asyncio.run(main())
```

(The server reads `DEEP_RESEARCH_TIMEOUT_SECONDS` from its environment via `load_config`, so passing a tiny value through `StdioServerParameters(env=...)` forces the timeout path deterministically.)

**Expected output** (real model):

```
case 1 (trace + summary) -> OK
case 2 (timeout -> PARTIAL) -> OK
```

That's the whole service: a fact-checked Researcher/Fact-Checker pipeline, capped at a fixed number of rounds, exposed as one `deep_research` MCP tool, reporting its progress as a trace, and always returning *something* — a full answer, or a clearly-marked partial — instead of hanging.